

Jeremy Coleman

07/30/2026

Agentic Clinic Analyst

Overview and Industry Context

Healthcare clinics generate a lot of data just running day to day. Admissions, billing, insurance, demographics. Most small and mid-size clinics don't have anyone

on staff who can dig into that data, and hiring an analyst or buying a BI platform

isn't realistic for a lot of them.

This project builds an agent that does a first pass of that analysis. You give it a CSV of clinic data and one sentence about what you want to know, and it reads the

data, decides what's worth looking at, runs the analysis, and explains what it found

in plain language. It's not a replacement for a real analyst. It's a starting point for a clinic that doesn't have one.

Healthcare data also comes with a real constraint: patient records can't be published

openly. Anything built on public data has to use de-identified data under an agreement, or synthetic data. That constraint shaped this project and is covered more

in the Evaluation section.

Overview of the Integrated Solution

The system is a Jupyter notebook that runs an agent start to finish. You give it a CSV and one sentence about what you want to understand, and it reads the file, works

out what's in it, and decides on its own how many analyses to run, between two and

four.

For each one it picks a chart type, bar, line, or scatter, and which columns to compare. It runs the chart, reads the numbers back, and decides what to look at next

based on what it just found. When it's done, it writes a paragraph per chart and one

summary tying everything back to the original question.

Gemini handles the reasoning and the writing. Pandas and matplotlib handle the actual

data work. None of the analyses are scripted ahead of time, the agent decides, and a

different dataset would produce a different result.

Integration Rationale and Prior Projects

This project draws on four earlier capstones.

AI Programming Foundations, from SentimentScope (IMDB-sentiment-analyzer), a

transformer-based sentiment classifier trained from scratch in PyTorch (Paszke et al., 2019).

No code is reused, but the same foundation, clean functions, and a notebook someone

else can follow, made this project possible.

Conduct a Statistical Analysis Using Python, from the Burmese python survey analysis

(FL-python-analysis), using real USGS field data from 1995-2021. The same approach

carries over directly: check for nulls and duplicates first, check data types before

assuming a column can be grouped, and only draw conclusions the data supports.

Design of Autonomous and Semi-Autonomous Agentic Workflows, from the report writer

(agentic-report-writer), the source of the Thought, Action, Observation loop. That

project had a fixed number of searches built into the code, so the model never actually chose the count. This project fixed that. The cap is enforced in code, but

the agent decides how many analyses to run within it.

Generative AI Applications, from the Grimm's Fairy Tales transformer

(grimms-transformer). That model was trained from scratch to generate text one

character at a time, and even after training, it only produced something close to real

English words maybe half the time. That's how hard it is to get a model to generate

coherent language on your own hardware with a small dataset. There's no version of

that same effort scaled up to power an agent that reasons over data and decides what

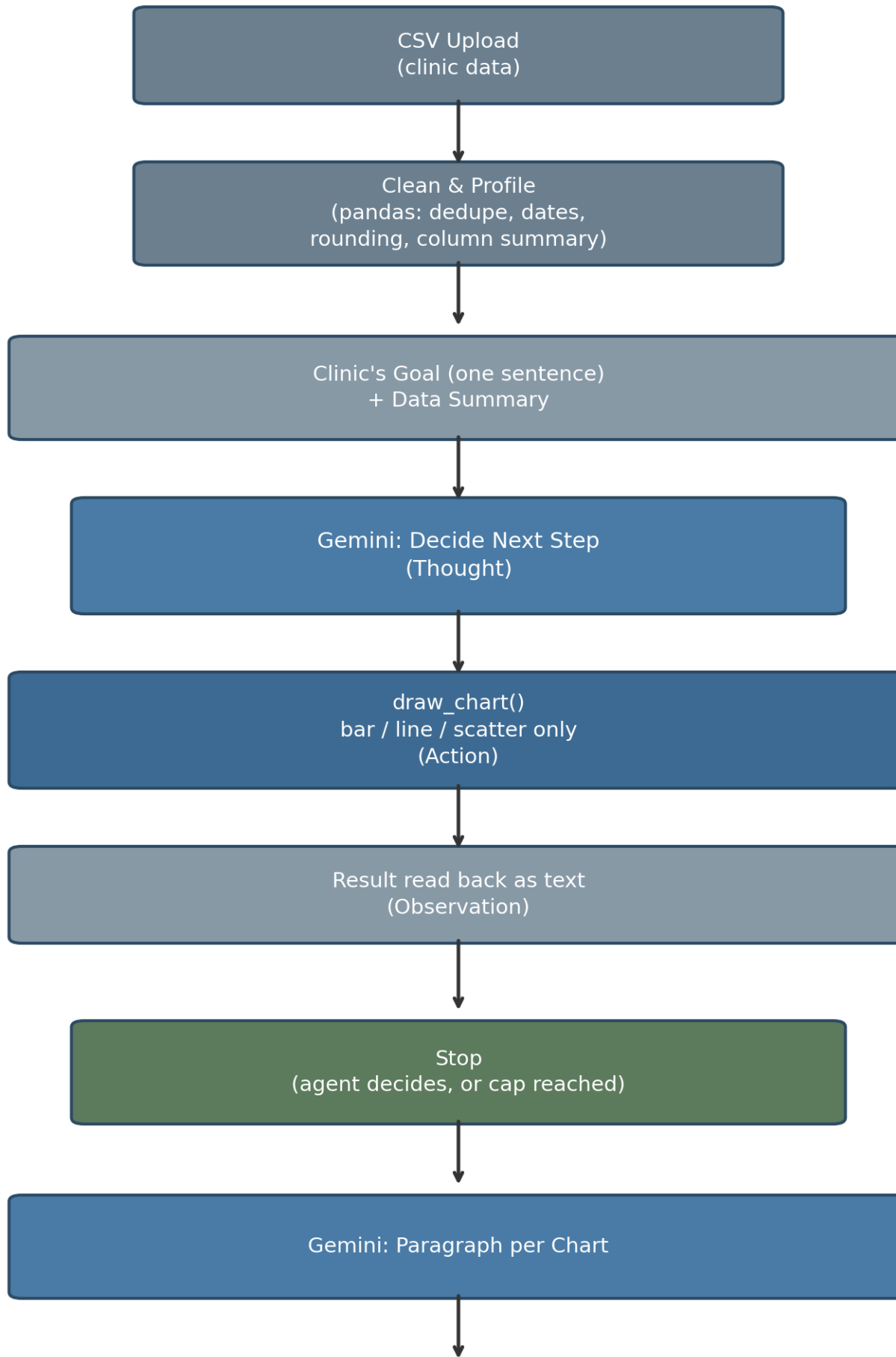
to analyze next, not without resources way past what this project had. That's the actual

reason this project uses Gemini through an API instead of training anything. It's not a

shortcut, it's knowing the difference between what you can build yourself and what

you need an existing model for.

System Design and Technical Decisions



The notebook runs a fixed setup, then a loop the agent controls.

First the CSV gets cleaned: duplicates dropped, dates converted from text to real

dates, billing rounded to two decimals. Then a function turns the cleaned data into a

written summary, row count, columns, types, and value counts, since the agent can't

see the CSV directly. That summary is handed to the agent before the loop starts, it's

not something the agent goes and fetches itself.

From there the agent takes over. Each pass it gets the data summary, the clinic's

goal, and a log of what's already been tried. It explains its reasoning, picks a chart

type and columns, and that chart runs. The result comes back as text and gets added

to the history before the next pass. This Thought, Action, Observation structure follows

the ReAct pattern (Yao et al., 2023).

Two things are locked down. Chart type can only be bar, line, or scatter, nothing

else runs. And the loop is capped at four analyses in the code itself, not just in the

prompt. Early on, before that cap was enforced in code, the agent ran five analyses

even though it had been told to stop at four. That's why the cap moved into the code.

Once it's done, Gemini writes a paragraph on each chart and one summary answering the

clinic's original question.

Tradeoffs and Constraints

The data summary is handed to the agent all at once rather than fetched piece by

piece. The dataset is small enough to describe in one pass, and there's only one

dataset here to request information about anyway.

Chart type is limited to bar, line, and scatter. Those are the three types whose results can be summarized in plain language without guessing at what they mean.

Analyses are capped at four, which keeps the output readable and stops the loop from

running indefinitely. A richer dataset might have supported more, and this system

would stop short of that.

The dataset itself is synthetic, since real patient-level data can't be published to

a public repository. That's covered more in the next section, but it's a real tradeoff on what the system could discover, not just how it was built.

Evaluation, Limitations, and Risks

I ran this twice. Both times every chart came back nearly flat, billing stayed within

a few hundred dollars of \$25,500 no matter what it was grouped by, and admission

counts split almost evenly across categories. That's what random data looks like, and

it happened the same way both runs.

The agent handled it right. It said plainly that nothing stood out instead of forcing

a pattern that wasn't there.

One real bug came out of testing. Before the four-chart cap was in the code, one run

went to five even though the prompt said stop at four. That's why I moved the cap into

the code.

The system's never been tested on data that actually has a pattern in it.
Staying

honest on flat data is one thing. Finding a real signal is a different test, and this hasn't faced that one yet.

Ethical Considerations and Responsible AI

The biggest risk here is that the person using this has no data background, and neither does the agent, really. Neither one can tell a real pattern from noise.
This

isn't a new problem, doctors and nurses do the same thing with AI tools all the time,

trust the output without questioning it, even when it's wrong (Khera et al., 2023).

This project actually ran into it. Every chart came back flat, and the agent was honest about that instead of forcing some conclusion that wasn't there. But nothing

stops someone from reading a real pattern into random noise anyway, or ignoring a real

signal because the last few charts happened to be flat. The agent doesn't check

whether anything is statistically significant, it just reports what it sees.

Two things help. It can only use bar, line, and scatter charts, which are easy to

explain honestly. And it's told to say plainly when nothing stands out instead of

making something up. But neither of those tells the clinic how much to trust a result.

This should be a starting point for a conversation with someone who actually knows

data, not something to act on by itself. This system flags what might be worth a closer

look. It's not a substitute for a professional.

Professional and Industry Relevance

This project shows a real skill set, building a system that makes its own decisions

instead of just running a fixed script, and doing it in a way that's honest about what

it doesn't know. That's the harder version of the job. Anyone can write code that

prints a chart. Writing something that decides which chart to make, checks its own

work, and admits when the data doesn't support a conclusion is closer to what an

actual data or AI role looks like.

It also shows something specific to working with limited information, which is most

real jobs. A small clinic isn't going to have a clean, well-labeled dataset with an obvious story in it. This project was built and tested on exactly that kind of messy, flat, inconclusive data, and the system held up honestly instead of pretending to find something that wasn't there.

Future Extensions or Improvements

The clearest next step is running this on data that actually has variation in it, something where cost or outcome really does differ by category, to see whether the agent can find and explain a real pattern, not just report the absence of one.

A few other things would make this more useful for an actual clinic. Some kind of confidence check before a finding gets reported, so it's not just numbers with no sense of how solid they are. A simple interface instead of a notebook, since nobody running a clinic is going to open Jupyter. And letting the agent pull in more than one file at a time, since real clinic data is usually split across several systems, not sitting in one CSV.

References

Khera, R., Simon, M. A., & Ross, J. S. (2023). Automation Bias and Assistive AI: Risk

of Harm From AI-Driven Clinical Decision Support. JAMA, 330(23), 2255-2257.

<https://doi.org/10.1001/jama.2023.22557>

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023).

ReAct: Synergizing Reasoning and Acting in Language Models.

arXiv:2210.03629.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library.

Advances in Neural Information Processing Systems, 32.